Uber-Like Ridesharing Database Architecture

Table of Contents

Overview

A ridesharing platform is a study in real-time, geospatial data management. The database must handle continuous location updates from millions of drivers, sub-second matching of riders to drivers, dynamic pricing computation, and a permanent transactional record for every trip. This document models the PostgreSQL schema with PostGIS for geospatial operations and discusses when to delegate location tracking to purpose-built systems.

Requirements

Functional Requirements

- Riders request trips from a pickup to a destination
- Drivers go online/offline and receive trip requests
- Real-time driver location tracking (updates every 4 seconds)
- Matching engine finds the nearest available driver
- Dynamic surge pricing based on supply/demand ratio
- Fare estimation before booking; final fare after trip
- Multiple vehicle classes: economy, premium, XL, pet-friendly
- Rating system: riders rate drivers and vice versa
- Multiple payment methods; automatic charge after trip
- Driver earnings and payout management
- Trip receipts and detailed history

Non-Functional Requirements

- 5M+ drivers globally
- 15M+ daily trip requests
- Location updates: up to 1.5M writes/second at peak
- Sub-500ms driver matching
- 99.99% uptime for trip request and payment flows
- GDPR: right to erasure for rider personal data

- Location data retained 30 days for operational use; 2 years anonymized

---

Capacity Estimation

Metric	Estimate
Registered drivers	5M
Registered riders	100M
Active drivers (peak hour)	800K
Daily trips	15M
Peak trips per second	500
Location updates per second	1.5M
Average trip duration	18 minutes
Average fare	$12

Storage Estimates

Table	Daily Rows	Row Size	Annual Storage
trips	15M	600B	~3.3TB/yr
trip_events	150M	200B	~11TB/yr
driver_locations	1.5M writes/s × 86400	100B	offloaded
payments	15M	400B	~2.2TB/yr
ratings	30M	150B	~1.6TB/yr

Key insight: Raw location updates (1.5M/s) are NOT stored in PostgreSQL — they go to Redis/Cassandra. Only the trip path (sampled every 30s) is stored in PostgreSQL.

Schema Design

```
-- ============================================================
-- EXTENSIONS
-- ============================================================
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "postgis";           -- geospatial
CREATE EXTENSION IF NOT EXISTS "postgis_topology";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- ============================================================
-- ENUM TYPES
-- ============================================================
CREATE TYPE user_status AS ENUM ('active', 'suspended', 'deleted');
```

```
CREATE TYPE driver_status AS ENUM ('offline', 'available', 'on_trip', 'in_break');
CREATE TYPE vehicle_class AS ENUM ('economy', 'comfort', 'premium', 'xl', 'pet',
'accessibility');
CREATE TYPE trip_status AS ENUM (
    'requested', 'searching', 'driver_assigned', 'driver_en_route',
    'arrived', 'in_progress', 'completed', 'cancelled', 'no_driver_found'
);
CREATE TYPE cancel_actor AS ENUM ('rider', 'driver', 'system');
CREATE TYPE payment_status AS ENUM ('pending', 'authorized', 'captured', 'failed',
'refunded');
CREATE TYPE surge_reason AS ENUM ('high_demand', 'low_supply', 'event', 'weather');

-- ============================================================
-- USERS (RIDERS)
-- ============================================================
CREATE TABLE riders (
    rider_id        UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    email           VARCHAR(320)    NOT NULL,
    phone           VARCHAR(30)     NOT NULL,
    full_name       VARCHAR(200)    NOT NULL,
    status          user_status     NOT NULL DEFAULT 'active',
    avg_rating      NUMERIC(3,2)    NOT NULL DEFAULT 5.0,
    rating_count    INT             NOT NULL DEFAULT 0,
    default_pm_id   UUID,           -- FK added after payment_methods
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_riders_email UNIQUE (email),
    CONSTRAINT uq_riders_phone UNIQUE (phone)
);

CREATE TABLE rider_payment_methods (
    pm_id           UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    rider_id        UUID            NOT NULL REFERENCES riders(rider_id) ON DELETE
CASCADE,
    provider        VARCHAR(50)     NOT NULL,
    token           VARCHAR(500)    NOT NULL,
    last_four       CHAR(4),
    expiry_month    SMALLINT,
    expiry_year     SMALLINT,
    is_default      BOOLEAN         NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

ALTER TABLE riders ADD CONSTRAINT fk_riders_default_pm
    FOREIGN KEY (default_pm_id) REFERENCES rider_payment_methods(pm_id)
    DEFERRABLE INITIALLY DEFERRED;

-- ============================================================
-- DRIVERS
-- ============================================================
CREATE TABLE drivers (
    driver_id       UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    email           VARCHAR(320)    NOT NULL,
    phone           VARCHAR(30)     NOT NULL,
    full_name       VARCHAR(200)    NOT NULL,
    license_number  VARCHAR(50)     NOT NULL,
    license_expiry  DATE            NOT NULL,
    status          driver_status   NOT NULL DEFAULT 'offline',
    avg_rating      NUMERIC(3,2)    NOT NULL DEFAULT 5.0,
    rating_count    INT             NOT NULL DEFAULT 0,
    acceptance_rate NUMERIC(5,4),
    completion_rate NUMERIC(5,4),
    total_trips     INT             NOT NULL DEFAULT 0,
    city_id         INT,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_drivers_email UNIQUE (email),
    CONSTRAINT uq_drivers_phone UNIQUE (phone),
    CONSTRAINT uq_drivers_license UNIQUE (license_number)
);

CREATE TABLE vehicles (
    vehicle_id      UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    driver_id       UUID            NOT NULL REFERENCES drivers(driver_id),
    class           vehicle_class   NOT NULL,
    make            VARCHAR(100)    NOT NULL,
    model           VARCHAR(100)    NOT NULL,
    year            SMALLINT        NOT NULL,
    color           VARCHAR(50),
    plate_number    VARCHAR(20)     NOT NULL,
    vin             VARCHAR(17),
    capacity        SMALLINT        NOT NULL DEFAULT 4,
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    inspected_at    DATE,
    inspection_due  DATE,
    CONSTRAINT uq_vehicles_plate UNIQUE (plate_number)
);

-- Driver's current location — updated every 4 seconds by active drivers
-- This is the REAL-TIME table; historical locations are NOT stored here
CREATE TABLE driver_locations (
    driver_id       UUID            PRIMARY KEY REFERENCES drivers(driver_id),
    location        GEOGRAPHY(POINT, 4326) NOT NULL,   -- PostGIS geography
    heading         SMALLINT,       -- degrees 0-359
    speed_kmh       SMALLINT,
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- ============================================================
-- GEOSPATIAL ZONES
-- ============================================================
CREATE TABLE cities (
    city_id         SERIAL          PRIMARY KEY,
    name            VARCHAR(200)    NOT NULL,
```

```
    country_code    CHAR(2)         NOT NULL,
    timezone        VARCHAR(60)     NOT NULL,
    boundary        GEOGRAPHY(MULTIPOLYGON, 4326),
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE
);

CREATE TABLE zones (
    zone_id         SERIAL          PRIMARY KEY,
    city_id         INT             NOT NULL REFERENCES cities(city_id),
    name            VARCHAR(200)    NOT NULL,
    boundary        GEOGRAPHY(POLYGON, 4326) NOT NULL,
    zone_type       VARCHAR(50)     NOT NULL DEFAULT 'standard'  -- 'airport',
'stadium', 'cbd'
);

-- ============================================================
-- PRICING
-- ============================================================
CREATE TABLE pricing_rules (
    rule_id         SERIAL          PRIMARY KEY,
    city_id         INT             NOT NULL REFERENCES cities(city_id),
    vehicle_class   vehicle_class   NOT NULL,
    base_fare       NUMERIC(8,2)    NOT NULL,
    cost_per_km     NUMERIC(8,4)    NOT NULL,
    cost_per_minute NUMERIC(8,4)    NOT NULL,
    min_fare        NUMERIC(8,2)    NOT NULL,
    booking_fee     NUMERIC(8,2)    NOT NULL DEFAULT 0,
    valid_from      DATE            NOT NULL,
    valid_until     DATE,
    CONSTRAINT uq_pricing UNIQUE (city_id, vehicle_class, valid_from)
);

-- Surge pricing state — updated by pricing microservice every minute
CREATE TABLE surge_pricing (
    zone_id         INT             NOT NULL REFERENCES zones(zone_id),
    vehicle_class   vehicle_class   NOT NULL,
    multiplier      NUMERIC(4,2)    NOT NULL DEFAULT 1.0,
    reason          surge_reason,
    demand_index    NUMERIC(5,2),   -- riders requesting per hour
    supply_index    NUMERIC(5,2),   -- available drivers in zone
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    PRIMARY KEY (zone_id, vehicle_class),
    CONSTRAINT chk_multiplier CHECK (multiplier BETWEEN 1.0 AND 8.0)
);

-- ============================================================
-- TRIPS
-- ============================================================
CREATE TABLE trips (
    trip_id         UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    rider_id        UUID            NOT NULL REFERENCES riders(rider_id),
    driver_id       UUID            REFERENCES drivers(driver_id),
```

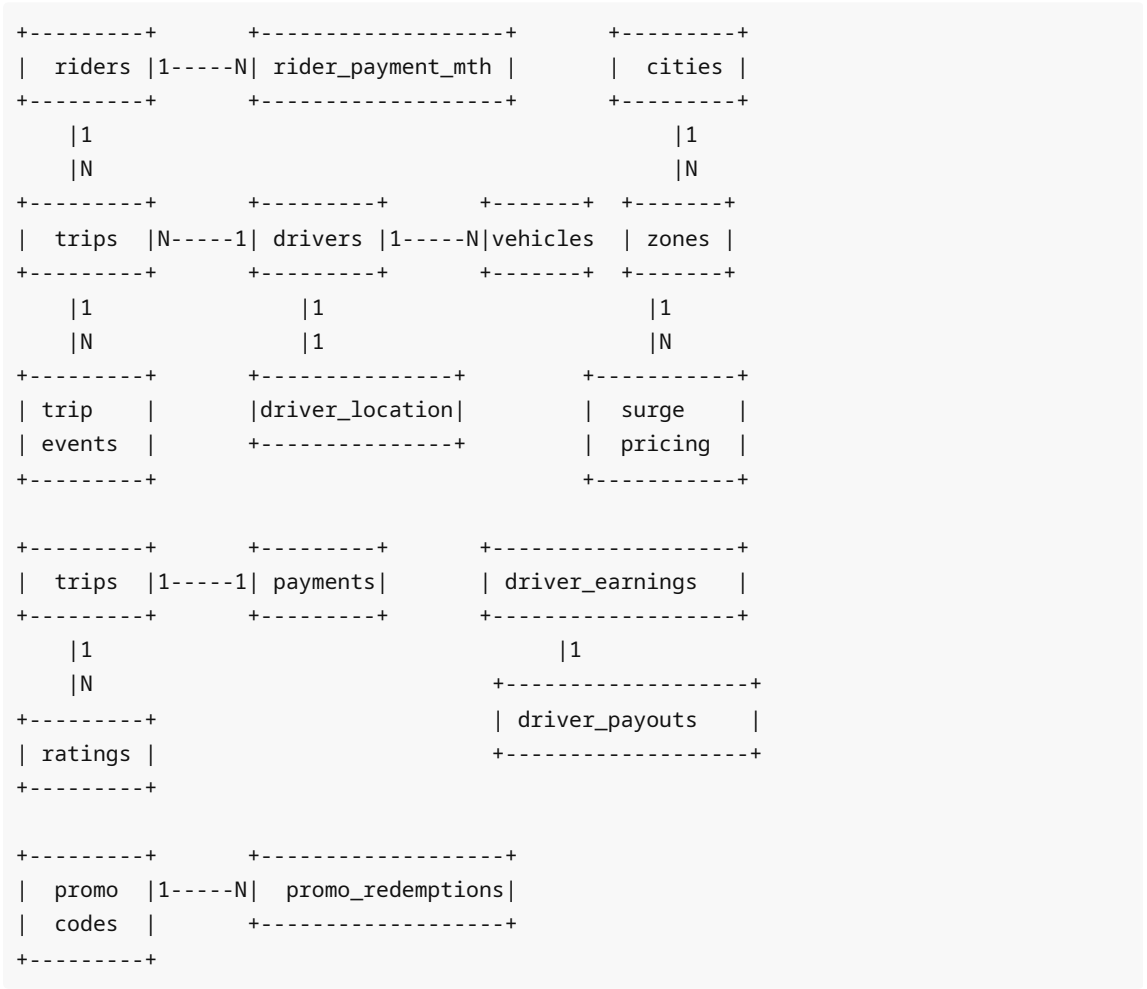
```
    vehicle_id      UUID            REFERENCES vehicles(vehicle_id),
    vehicle_class   vehicle_class   NOT NULL,
    status          trip_status     NOT NULL DEFAULT 'requested',

    -- Pickup
    pickup_location         GEOGRAPHY(POINT, 4326) NOT NULL,
    pickup_address          VARCHAR(500),
    pickup_zone_id          INT             REFERENCES zones(zone_id),

    -- Destination
    destination_location    GEOGRAPHY(POINT, 4326),
    destination_address     VARCHAR(500),
    destination_zone_id     INT             REFERENCES zones(zone_id),

    -- Actual route (recorded when trip ends)
    actual_route            GEOGRAPHY(LINESTRING, 4326),
    distance_km             NUMERIC(8,3),
    duration_minutes        NUMERIC(8,2),

    -- Timing
    requested_at            TIMESTAMPTZ     NOT NULL DEFAULT now(),
    driver_assigned_at      TIMESTAMPTZ,
    driver_arrived_at       TIMESTAMPTZ,
    trip_started_at         TIMESTAMPTZ,
    trip_ended_at           TIMESTAMPTZ,
    cancelled_at            TIMESTAMPTZ,

    -- Pricing
    estimated_fare          NUMERIC(10,2),
    surge_multiplier        NUMERIC(4,2)    NOT NULL DEFAULT 1.0,
    base_fare               NUMERIC(10,2),
    distance_fare           NUMERIC(10,2),
    time_fare               NUMERIC(10,2),
    booking_fee             NUMERIC(10,2)   NOT NULL DEFAULT 0,
    promo_discount          NUMERIC(10,2)   NOT NULL DEFAULT 0,
    final_fare              NUMERIC(10,2),
    currency                CHAR(3)         NOT NULL DEFAULT 'USD',

    -- Cancellation
    cancel_actor            cancel_actor,
    cancel_reason           TEXT,
    cancel_fee              NUMERIC(10,2)   NOT NULL DEFAULT 0,

    -- Search metadata
    search_radius_meters    INT,
    drivers_considered      SMALLINT,

    created_at              TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at              TIMESTAMPTZ     NOT NULL DEFAULT now()
) PARTITION BY RANGE (requested_at);

CREATE TABLE trips_2024_q1 PARTITION OF trips
```

```
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
CREATE TABLE trips_2024_q2 PARTITION OF trips
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
CREATE TABLE trips_2024_q3 PARTITION OF trips
    FOR VALUES FROM ('2024-07-01') TO ('2024-10-01');
CREATE TABLE trips_2024_q4 PARTITION OF trips
    FOR VALUES FROM ('2024-10-01') TO ('2025-01-01');

-- Trip state machine audit log
CREATE TABLE trip_events (
    event_id        UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    trip_id         UUID            NOT NULL REFERENCES trips(trip_id),
    status          trip_status     NOT NULL,
    actor           VARCHAR(50),    -- 'rider', 'driver', 'system'
    metadata        JSONB,
    location        GEOGRAPHY(POINT, 4326),
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
) PARTITION BY RANGE (created_at);

-- ============================================================
-- PAYMENTS
-- ============================================================
CREATE TABLE payments (
    payment_id      UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    trip_id         UUID            NOT NULL REFERENCES trips(trip_id),
    rider_id        UUID            NOT NULL REFERENCES riders(rider_id),
    pm_id           UUID            REFERENCES rider_payment_methods(pm_id),
    amount          NUMERIC(10,2)   NOT NULL,
    currency        CHAR(3)         NOT NULL DEFAULT 'USD',
    status          payment_status  NOT NULL DEFAULT 'pending',
    provider        VARCHAR(50)     NOT NULL,
    provider_txn_id VARCHAR(200),
    authorized_at   TIMESTAMPTZ,
    captured_at     TIMESTAMPTZ,
    failed_at       TIMESTAMPTZ,
    failure_reason  TEXT,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_payments_trip UNIQUE (trip_id)
);

-- Driver earnings ledger
CREATE TABLE driver_earnings (
    earning_id      UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    driver_id       UUID            NOT NULL REFERENCES drivers(driver_id),
    trip_id         UUID            NOT NULL REFERENCES trips(trip_id),
    gross_fare      NUMERIC(10,2)   NOT NULL,
    platform_fee    NUMERIC(10,2)   NOT NULL,
    net_earning     NUMERIC(10,2)   NOT NULL,
    currency        CHAR(3)         NOT NULL DEFAULT 'USD',
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_driver_earnings_trip UNIQUE (trip_id)
);
```

```
CREATE TABLE driver_payouts (
    payout_id       UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    driver_id       UUID            NOT NULL REFERENCES drivers(driver_id),
    amount          NUMERIC(12,2)   NOT NULL,
    currency        CHAR(3)         NOT NULL DEFAULT 'USD',
    period_start    DATE            NOT NULL,
    period_end      DATE            NOT NULL,
    bank_account    VARCHAR(500),   -- tokenized bank reference
    provider_payout_id VARCHAR(200),
    status          VARCHAR(30)     NOT NULL DEFAULT 'pending',
    initiated_at    TIMESTAMPTZ,
    completed_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- ============================================================
-- RATINGS
-- ============================================================
CREATE TABLE ratings (
    rating_id       UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    trip_id         UUID            NOT NULL REFERENCES trips(trip_id),
    from_rider      BOOLEAN         NOT NULL,   -- TRUE=rider rating driver,
FALSE=driver rating rider
    rated_by        UUID            NOT NULL REFERENCES users_lookup(user_id),
    target_id       UUID            NOT NULL,   -- driver_id or rider_id
    score           SMALLINT        NOT NULL,
    comment         TEXT,
    tags            TEXT[],         -- ['clean_car','friendly','music']
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_rating_per_trip_direction UNIQUE (trip_id, from_rider),
    CONSTRAINT chk_rating_score CHECK (score BETWEEN 1 AND 5)
);

-- Unified user lookup for ratings FK
CREATE TABLE users_lookup (
    user_id         UUID            PRIMARY KEY,
    user_type       VARCHAR(10)     NOT NULL    -- 'rider' or 'driver'
);
-- Populated by trigger on INSERT to riders/drivers

-- ============================================================
-- PROMO CODES
-- ============================================================
CREATE TABLE promo_codes (
    promo_id        UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    code            VARCHAR(50)     NOT NULL,
    discount_type   VARCHAR(20)     NOT NULL,   -- 'percent', 'fixed'
    discount_value  NUMERIC(10,2)   NOT NULL,
    max_discount    NUMERIC(10,2),
    min_fare        NUMERIC(10,2),
    max_uses        INT,
```

```
    uses_per_user   SMALLINT        NOT NULL DEFAULT 1,
    current_uses    INT             NOT NULL DEFAULT 0,
    valid_from      TIMESTAMPTZ     NOT NULL,
    valid_until     TIMESTAMPTZ,
    city_id         INT             REFERENCES cities(city_id),
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_promo_code UNIQUE (code)
);

CREATE TABLE promo_redemptions (
    redemption_id   UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    promo_id        UUID            NOT NULL REFERENCES promo_codes(promo_id),
    rider_id        UUID            NOT NULL REFERENCES riders(rider_id),
    trip_id         UUID            REFERENCES trips(trip_id),
    discount_amount NUMERIC(10,2)   NOT NULL,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_promo_redemption UNIQUE (promo_id, rider_id, trip_id)
);
```

ASCII ER Diagram



Indexing Strategy

```sql
-- Geospatial: driver location queries (most critical index)
CREATE INDEX idx_driver_locations_geography
    ON driver_locations USING GIST (location);

-- Filter by availability status before geospatial search
CREATE INDEX idx_drivers_status
    ON drivers (status) WHERE status = 'available';

-- Zone boundary containment queries
CREATE INDEX idx_zones_boundary
    ON zones USING GIST (boundary);
CREATE INDEX idx_cities_boundary
    ON cities USING GIST (boundary);

-- Trips: rider history
CREATE INDEX idx_trips_rider
    ON trips (rider_id, requested_at DESC);

-- Trips: driver history
CREATE INDEX idx_trips_driver
    ON trips (driver_id, requested_at DESC)
    WHERE driver_id IS NOT NULL;

-- Trips: active trip lookup (sub-second for dispatch service)
CREATE INDEX idx_trips_active
    ON trips (status, requested_at)
    WHERE status NOT IN ('completed', 'cancelled', 'no_driver_found');

-- Trip events: audit log query
CREATE INDEX idx_trip_events_trip
    ON trip_events (trip_id, created_at ASC);

-- Payments: reconciliation
CREATE INDEX idx_payments_status
    ON payments (status, created_at)
    WHERE status IN ('pending', 'authorized');

-- Driver earnings: payout computation
CREATE INDEX idx_driver_earnings_driver
    ON driver_earnings (driver_id, created_at DESC);

-- Surge pricing: zone lookup (hot read)
CREATE INDEX idx_surge_zone
    ON surge_pricing (zone_id, vehicle_class);

-- Ratings: compute driver/rider aggregate
CREATE INDEX idx_ratings_target
    ON ratings (target_id, score);
```

Query Patterns

Q1 — Find Nearest Available Drivers (Matching — critical hot path)

```
-- Uses PostGIS ST_DWithin for bounding box pre-filter + distance sort
SELECT
    d.driver_id,
    v.vehicle_id,
    v.class,
    d.avg_rating,
    ST_Distance(dl.location::geography, ST_Point($lng, $lat)::geography) AS
distance_meters,
    dl.heading,
    dl.updated_at
FROM driver_locations dl
JOIN drivers d ON d.driver_id = dl.driver_id
JOIN vehicles v ON v.driver_id = d.driver_id AND v.is_active
WHERE d.status = 'available'
  AND v.class = $vehicle_class
  AND ST_DWithin(
      dl.location::geography,
      ST_Point($lng, $lat)::geography,
      5000              -- 5km search radius in meters
  )
  AND dl.updated_at > now() - INTERVAL '30 seconds'  -- stale location filter
ORDER BY distance_meters ASC
LIMIT 20;
```

Q2 — Estimate Fare

```
WITH pricing AS (
    SELECT pr.base_fare, pr.cost_per_km, pr.cost_per_minute, pr.booking_fee,
pr.min_fare
    FROM pricing_rules pr
    WHERE pr.city_id = $city_id
      AND pr.vehicle_class = $vehicle_class
      AND pr.valid_from <= CURRENT_DATE
      AND (pr.valid_until IS NULL OR pr.valid_until >= CURRENT_DATE)
    ORDER BY pr.valid_from DESC LIMIT 1
),
surge AS (
    SELECT COALESCE(sp.multiplier, 1.0) AS multiplier
    FROM surge_pricing sp
    WHERE sp.zone_id = $pickup_zone_id
      AND sp.vehicle_class = $vehicle_class
)
SELECT
    p.base_fare,
    p.cost_per_km * $distance_km AS distance_component,
```

```
    p.cost_per_minute * $duration_min AS time_component,
    s.multiplier,
    GREATEST(
        (p.base_fare + p.cost_per_km * $distance_km + p.cost_per_minute *
$duration_min)
        * s.multiplier + p.booking_fee,
        p.min_fare
    ) AS estimated_fare
FROM pricing p, surge s;
```

Q3 — Update Driver Location (highest frequency write)

```
INSERT INTO driver_locations (driver_id, location, heading, speed_kmh, updated_at)
VALUES (
    $driver_id,
    ST_SetSRID(ST_MakePoint($lng, $lat), 4326),
    $heading,
    $speed,
    now()
)
ON CONFLICT (driver_id) DO UPDATE
    SET location   = EXCLUDED.location,
        heading    = EXCLUDED.heading,
        speed_kmh  = EXCLUDED.speed_kmh,
        updated_at = EXCLUDED.updated_at;
```

Q4 — Create Trip (transactional)

```
BEGIN;

INSERT INTO trips (
    trip_id, rider_id, vehicle_class,
    pickup_location, pickup_address, pickup_zone_id,
    destination_location, destination_address, destination_zone_id,
    estimated_fare, surge_multiplier, status
) VALUES (
    uuid_generate_v4(), $rider_id, $vehicle_class,
    ST_SetSRID(ST_MakePoint($p_lng, $p_lat), 4326), $pickup_addr, $pickup_zone,
    ST_SetSRID(ST_MakePoint($d_lng, $d_lat), 4326), $dest_addr, $dest_zone,
    $est_fare, $surge_mult, 'requested'
) RETURNING trip_id;

INSERT INTO trip_events (trip_id, status, actor)
VALUES ($trip_id, 'requested', 'rider');

COMMIT;
```

Q5 — Assign Driver to Trip

```
BEGIN;

UPDATE trips
SET status = 'driver_assigned',
    driver_id = $driver_id,
    vehicle_id = $vehicle_id,
    driver_assigned_at = now(),
    updated_at = now()
WHERE trip_id = $trip_id
  AND status = 'requested';   -- Optimistic lock: only assign if still open

UPDATE drivers
SET status = 'on_trip', updated_at = now()
WHERE driver_id = $driver_id
  AND status = 'available';

INSERT INTO trip_events (trip_id, status, actor, metadata)
VALUES ($trip_id, 'driver_assigned', 'system',
        jsonb_build_object('driver_id', $driver_id));

COMMIT;
```

Q6 — Complete Trip and Calculate Fare

```
BEGIN;

UPDATE trips
SET status            = 'completed',
    trip_ended_at     = now(),
    distance_km       = $distance_km,
    duration_minutes  = $duration_min,
    base_fare         = $base_fare,
    distance_fare     = $dist_fare,
    time_fare         = $time_fare,
    final_fare        = $final_fare,
    actual_route      = ST_GeomFromText($route_wkt, 4326),
    updated_at        = now()
WHERE trip_id = $trip_id;

UPDATE drivers
SET status = 'available', total_trips = total_trips + 1
WHERE driver_id = $driver_id;

INSERT INTO driver_earnings (driver_id, trip_id, gross_fare, platform_fee,
net_earning)
VALUES ($driver_id, $trip_id, $final_fare, $platform_fee, $net_earning);

INSERT INTO payments (trip_id, rider_id, pm_id, amount, status, provider)
VALUES ($trip_id, $rider_id, $pm_id, $final_fare, 'pending', $provider);
```

```sql
INSERT INTO trip_events (trip_id, status, actor)
VALUES ($trip_id, 'completed', 'system');

COMMIT;
```

Q7 — Rider Trip History

```sql
SELECT
    t.trip_id, t.status, t.requested_at, t.trip_ended_at,
    t.pickup_address, t.destination_address,
    t.final_fare, t.currency, t.vehicle_class,
    t.distance_km, t.duration_minutes,
    d.full_name AS driver_name, d.avg_rating AS driver_rating,
    v.make, v.model, v.plate_number,
    r.score AS my_rating
FROM trips t
LEFT JOIN drivers d ON d.driver_id = t.driver_id
LEFT JOIN vehicles v ON v.vehicle_id = t.vehicle_id
LEFT JOIN ratings r ON r.trip_id = t.trip_id AND r.from_rider = TRUE
WHERE t.rider_id = $1
ORDER BY t.requested_at DESC
LIMIT 20 OFFSET $2;
```

Q8 — Surge Pricing Heatmap (admin/analytics)

```sql
SELECT
    z.name AS zone_name,
    z.zone_id,
    sp.vehicle_class,
    sp.multiplier,
    sp.demand_index,
    sp.supply_index,
    sp.updated_at
FROM surge_pricing sp
JOIN zones z ON z.zone_id = sp.zone_id
WHERE z.city_id = $1
  AND sp.multiplier > 1.0
ORDER BY sp.multiplier DESC;
```

Q9 — Driver Earnings Summary

```sql
SELECT
    DATE_TRUNC('week', de.created_at) AS week,
    COUNT(*) AS trips,
    SUM(de.gross_fare) AS gross,
    SUM(de.platform_fee) AS fees,
    SUM(de.net_earning) AS net,
    de.currency
```

```
FROM driver_earnings de
WHERE de.driver_id = $1
  AND de.created_at >= NOW() - INTERVAL '12 weeks'
GROUP BY 1, 6
ORDER BY 1 DESC;
```

Q10 — Drivers Within Zone Boundary

```
SELECT d.driver_id, d.full_name, dl.location
FROM driver_locations dl
JOIN drivers d ON d.driver_id = dl.driver_id
JOIN zones z ON ST_Within(dl.location::geometry, z.boundary::geometry)
WHERE z.zone_id = $1
  AND d.status = 'available'
  AND dl.updated_at > now() - INTERVAL '30 seconds';
```

Scaling Strategy

Real-Time Location: The Core Problem

At 800K active drivers × 1 update/4s = 200K writes/second to `driver_locations`. PostgreSQL can handle ~5K–50K writes/second depending on hardware. Options:

Option A — PostgreSQL with UNLOGGED table (up to 50M drivers)

```
-- Make driver_locations unlogged (no WAL, loses data on crash — acceptable for ephemeral location)
ALTER TABLE driver_locations SET UNLOGGED;
```

Option B — Redis geospatial (recommended for production)

```
Driver App → GEOADD drivers:available {lng} {lat} {driver_id}
Matching query → GEORADIUS drivers:available {lat} {lng} 5km ASC COUNT 20
PostgreSQL stores: only final trip route, not every heartbeat
```

Option C — Purpose-built system (S2 Geometry + Cassandra at Uber scale)

Phase 1: City-Scale (1 city, 50K drivers)

- Single PostgreSQL + PostGIS
- Redis for live driver locations
- All tables in one database

Phase 2: Regional Scale (10 cities, 1M drivers)

- Shard trips by city (city_id modulo shards)
- driver_locations moved entirely to Redis geospatial
- Trips partitioned quarterly, old data to analytics warehouse
- Dedicated pricing service with Redis cache for surge state

Phase 3: Global Scale (100+ cities, 5M+ drivers)

- Geospatial service (S2 Geometry library) handles location indexing
- PostgreSQL per region for trip records and driver profiles
- Kafka events: LocationUpdated → SurgePricingService → DB
- CQRS: Write to PostgreSQL, read from Redis/ElasticSearch

Tradeoffs

Decision	Chosen	Alternative	Why
PostGIS GEOGRAPHY type	geography	geometry	Geography uses spherical calculations — more accurate for real-world distances
UNLOGGED driver_locations	UNLOGGED table	WAL-logged	Location data is ephemeral; losing it on crash is acceptable; 10x write throughput
Trip partitioned quarterly	Quarterly	Monthly	Trips are smaller volume than orders; quarterly keeps fewer partitions
Store final route as LINESTRING	One column	Route table	Simplifies queries; entire route rarely queried (only for dispute resolution)
Optimistic lock on trip assignment	WHERE status='requested'	SELECT FOR UPDATE	Avoids row-level lock; at high concurrency, re-query is cheaper

---

Interview Discussion Points

1. **How do you find the nearest driver in under 500ms?** Redis GEORADIUS returns nearest drivers in O(N+log(M)) where N is results and M is total indexed members. For 800K drivers globally but ~5K in a city, this is sub-millisecond. The result is then filtered by vehicle_class and freshness.

2. **How does surge pricing work?** Every 60 seconds, the pricing service computes demand/supply ratio per zone (requests in last 10 min / available drivers) and writes a multiplier to `surge_pricing`. This table is tiny (zones × classes) and fits entirely in PostgreSQL's shared_buffers.

3. **How do you handle the race condition when two trip requests match the same driver?** The `UPDATE drivers SET status='on_trip' WHERE driver_id=$1 AND status='available'` is atomic. The first request that executes this UPDATE wins. The second gets 0 rows updated and retries with the next candidate driver.

4. **Why store actual_route as LINESTRING?** For dispute resolution and fare recalculation only. Storing every GPS point (every 4 seconds for 18 minutes = 270 points per trip) would consume 270 × 40B × 15M trips/day = 162GB/day. The route LINESTRING with 20 sampled points is 1.6GB/day.

Common Interview Follow-ups

Q: How do you implement geofencing for airport pickups? A: Zones table has a `zone_type = 'airport'` and a POLYGON boundary. When a driver enters/exits, the ride-share app posts the location; the

backend does `ST_Within(location, zone.boundary)` to detect zone entry and applies airport surcharges from pricing_rules.

Q: How would you implement driver batching (UberPool)? A: Add `pool_trip_id` on trips. A pool coordinator service finds trips with overlapping routes (ST_HausdorffDistance < threshold), groups them, and assigns the same driver. Each rider's trip record references the pool_trip_id.

Q: How does fraud detection work? A: Flag trips where `distance_km` significantly diverges from the straight-line distance between pickup/destination (detour ratio), or where `duration_minutes` is anomalously low for the distance. Surface these to a fraud_flags table consumed by the review team.

Performance Considerations

- **driver_locations**: With UNLOGGED + GiST index, can sustain ~100K updates/second per PostgreSQL instance.
- **Partition pruning**: All trip queries include `requested_at` filter — PostgreSQL prunes 97%+ of partitions automatically.
- **Connection pooling**: Dispatch service makes thousands of location queries/second; PgBouncer in statement mode reduces connections from thousands to 50–100.
- **PostGIS index**: GiST index on geography columns gives O(log N) spatial searches vs O(N) sequential. Always check `EXPLAIN` for `Index Scan using idx_driver_locations_geography`.
- **fillfactor on driver_locations**: Set `fillfactor=50` — this table is 100% UPSERTs; leaving room for HOT updates eliminates index maintenance per update.

Cross-References

- See `04_banking_platform.md` for payment processing and double-entry patterns
- See `21_System_Design/04_ride_sharing_design.md` for full system architecture
- See `21_System_Design/08_system_design_framework.md` for interview framework
- PostGIS spatial indexing: Chapter 11 of this guide
- Table partitioning: Chapter 5 of this guide